

A SIX-MINUTE INTRODUCTION

From a question to a controlled action

One customer-support case explains AI agents, LangChain, LangGraph and the agent loop. No AI background required.

The customer's goal

Alice asks: **"Can I get a refund for order O1002?"**

She wants a correct answer and, if eligible, a way to submit a request.

What is an AI agent?

An AI agent is software that uses a language model to interpret a goal and help choose actions. It can use approved tools, examine their results and continue until it can answer, complete a task or ask a person for help.

What makes this more than a text answer?

A text-only model can explain a refund policy. A connected agent system can check Alice's actual order, keep track of the conversation, request permission and register a refund application through controlled software.

The boundary that matters

The model helps interpret the request. The application enforces permissions and business rules.

Submitting a refund request does not move money.

Scope note: NexusAgent currently runs a rule-based demo. Its model integration exists, but live-model performance has not been validated. The case below explains the design, not a claim of autonomous production operation.

THE USE CASE

Follow Alice's request

Illustrative English dialogue. The current interface and many responses are in Chinese; the business sequence is the same.

01 Ask about eligibility

Alice: "Can O1002 be refunded?"

The system checks the signed-in account and reads the order. O1002 is processing, so it can be considered for a refund request. Nothing is submitted.

02 Ask for the policy

Alice: "What is the confirmation policy?"

The system retrieves published policy text and shows its document version and source lines. The order database still decides actual eligibility.

03 Request an action

Alice: "Please apply for O1002."

The system creates a proposal tied to this user and order, then displays a confirmation card. It saves progress and waits; there is still no refund request.

04 Give explicit permission

Alice clicks the confirmation button.

The server records her decision, resumes the workflow and rechecks ownership, order status and expiry before writing the request.

05 Receive a result

Alice sees a saved application reference.

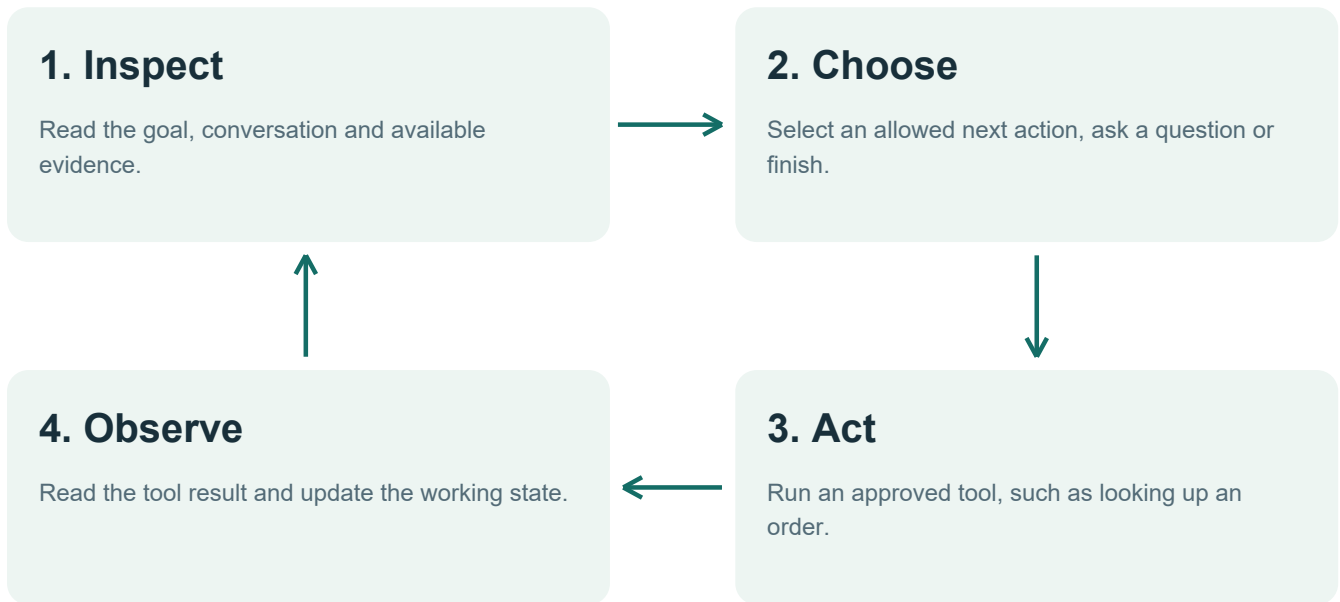
The request is registered, not paid out. Retrying the same confirmed operation returns the saved result rather than creating another application.

Other valid outcomes: an inaccessible or ineligible order is declined; cancellation creates no request; an expired confirmation requires a new proposal.

THE EXECUTION PATTERN

What is an agent loop?

A loop is repeated decision-making informed by new evidence. It is not simply generating a longer answer.



Repeat only if more work is needed. Stop to answer, wait for approval, or report a failure.

In Alice's case

The order lookup returns "processing". That evidence supports an eligibility answer. A later request to apply leads to a confirmation step, not immediate execution. Human permission becomes the next required input.

How NexusAgent uses this idea

Its active path is a **bounded, stateful workflow**: understand → resolve → approval → execute. Policy and inquiry paths can finish after resolve. It does not currently run an unrestricted model/tool loop.

The loop above describes the general agent pattern. NexusAgent deliberately fixes sensitive transitions in code. A new message or approval can start another saved workflow invocation. [3]

THE BUILDING BLOCKS

LangChain and LangGraph

Neither is the AI model. They are software building blocks around the model, with different responsibilities.

The language model: interpret the request

In model mode, it classifies Alice's message as an order inquiry, policy question, refund application or request needing clarification. It is not allowed to approve its own action.

LangChain: connect the model to the application

LangChain provides model integrations and agent-building interfaces. NexusAgent uses its model integration to obtain a structured intent, such as "refund" plus an order ID, rather than relying on unrestricted prose. [1]

LangGraph: control and remember the process

LangGraph is the workflow runtime. Nodes are steps; edges define where execution goes next; state is the saved case information. A checkpoint stores progress, so approval can pause execution and a later request can resume it. [2]

For Alice: model interpretation identifies her intent; LangChain connects that interpretation to code; LangGraph controls when the system checks, pauses and executes. PostgreSQL stores the business records and durable workflow checkpoints.

Relationship: LangChain's standard agents are built on LangGraph. This project uses a custom LangGraph workflow with a LangChain model adapter, not the standard create_agent loop. [1]

What makes an action safe?

The useful capability is controlled execution with evidence, permission and recovery. A fluent answer alone does not provide those guarantees.

Tool access is limited

A tool is a software function that reads data or performs a permitted action. Here, order access is scoped to the signed-in customer; a model cannot grant itself access to another account.

State is a saved case file

Conversation state remembers the current order and pending proposal. A checkpoint records workflow progress. This is stored application context, not a model learning or retraining itself.

Approval is a real gate

The confirmation is bound to the user, order and proposal, and has a 15-minute expiry. A statement such as "the user agreed" generated by a model is not authorization.

A retry does not mean a second refund

The server rechecks the order inside the execution path. Transactions and uniqueness constraints protect writes; request IDs and saved receipts support safe retries. This is the meaning of idempotency.

Operations can be inspected

A trace follows one request through its steps; metrics show counts, errors and durations; structured logs carry correlation IDs. Normal waiting is distinguished from failure, without collecting passwords or full prompts.

Knowledge retrieval means looking up approved text before answering. Current search is Chinese/English lexical matching, not vector-based semantic search; citations preserve the published version used.

What should a manager assess?

NexusAgent demonstrates the engineering controls around an agent-enabled service. Business impact still needs to be measured in a real pilot.

A three-minute demonstration

1. Ask about O1002 and inspect the policy source.
2. Request an application, then show the explicit confirmation gate.
3. Refresh while waiting, confirm, and inspect the saved result.
4. Retry the same action and show that no duplicate is created.
5. Open its trace to see where time was spent or a failure occurred.

What is delivered, and what is not yet proven?

Delivered: saved conversations, approval and recovery, transactional requests, versioned knowledge, citations and observability. The local demo uses deterministic rules with simulated orders. A live-model adapter exists; model quality, production scale and customer outcomes remain unvalidated. There is no payment-gateway integration.

How to judge a future pilot

Measure correct resolution rate, unauthorized-action attempts blocked, duplicate actions prevented, time per case, escalation rate and cost per resolved case. Compare against a human or existing workflow baseline. No time-saving, revenue or ROI improvement is claimed here.

The management takeaway

The model interprets. Tools access business systems. LangChain connects components. LangGraph manages the process. Your application retains authority over the action.

Sources and project evidence

[1] [LangChain overview](#)

[2] [LangGraph overview](#)

[3] [Workflows and agents](#)

Project evidence: README; docs/PROGRESS.md; backend/workflow.py, service.py and database.py. Framework definitions checked against official documentation. Dialogue is illustrative; scope reflects the repository snapshot.
